

แบบฝึกปฏิบัติในชั้นเรียน: Array, Linked List, Stack และ Queue

ข้อ 1: ระบบวิเคราะห์คะแนนนักศึกษา

หัวข้อ: Array 1 มิติ + Array 2 มิติ

กำหนดให้นักศึกษา 5 คน และแต่ละคนมีคะแนน 4 รายการ ได้แก่ Quiz, Assignment, Midterm และ Final

ตัวอย่างข้อมูล

```
int[][] scores = {  
    {8, 18, 25, 40},  
    {7, 15, 20, 35},  
    {10, 20, 28, 42},  
    {6, 16, 22, 30},  
    {9, 19, 27, 45}  
};
```

ให้นักศึกษาเขียนโปรแกรม Java เพื่อ

1. คำนวณคะแนนรวมของนักศึกษาแต่ละคน และเก็บไว้ใน Array 1 มิติ
2. หาคะแนนรวมสูงสุดและต่ำสุด
3. หาค่าเฉลี่ยของนักศึกษาทั้งห้อง
4. แสดงหมายเลขนักศึกษาที่มีคะแนนรวมสูงกว่าค่าเฉลี่ย
5. หาค่าเฉลี่ยของคะแนนแต่ละประเภท เช่น Quiz เฉลี่ยเท่าใด
6. วิเคราะห์ Time Complexity ของแต่ละส่วน

ตัวอย่างผลลัพธ์

Student 1 Total = 91

Student 2 Total = 77

Student 3 Total = 100

Student 4 Total = 74

Student 5 Total = 100

Average = 88.4

Highest = 100

Lowest = 74

Quiz Average = 8.0

Assignment Average = 17.6

Midterm Average = 24.4

Final Average = 38.4

Time Complexity: Total $O(nm)$, Highest/Lowest $O(n)$, Average $O(n)$, Above average $O(n)$, Average by type $O(nm)$

Students above average:

Student 1

Student 3

Student 5

คำถามวิเคราะห์

1. ถ้ามีนักศึกษา n คน และมีคะแนน m รายการต่อคน รวมข้อมูล = $n \times m$
2. การคำนวณคะแนนรวมมี Big-O เท่าใด? ตอบ: $O(nm)$
3. การหาค่าสูงสุดมี Big-O เท่าใด? ตอบ: $O(n)$
4. ถ้าค้นหาคะแนนของนักศึกษาคนหนึ่งด้วย index จะมี Big-O เท่าใด? ตอบ: $O(1)$

ข้อ 2: ระบบจัดการรายชื่อนักศึกษา 3 รูปแบบ

หัวข้อ: **Singly Linked List + Doubly Linked List + Circular Linked List**

ให้นักศึกษาออกแบบโปรแกรมจัดเก็บข้อมูลนักศึกษา

Student ID

Name

GPA

โดยสร้าง Linked List 3 แบบ ได้แก่

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

ให้ใช้ข้อมูลตัวอย่างอย่างน้อย 6 คน

นักศึกษาต้องสร้าง operation ต่อไปนี้

1. insertFirst()
2. insertLast()
3. search()
4. delete()
5. display()

จากนั้นเพิ่ม operation เฉพาะของแต่ละโครงสร้าง

Singly Linked List

ให้ค้นหานักศึกษาที่มี GPA สูงที่สุด

findMaxGPA();

Doubly Linked List

แสดงข้อมูลทั้งสองทิศทาง

Forward:

65001 -> 65002 -> 65003 -> 65004

Backward:

65004 -> 65003 -> 65002 -> 65001

Circular Linked List

แสดงข้อมูล 2 รอบเพื่อพิสูจน์ว่าโหนดสุดท้ายเชื่อมกลับไปยังโหนดแรก

65001 -> 65002 -> 65003 ->

65001 -> 65002 -> 65003

ส่วนวิเคราะห์

ให้นักศึกษาเติมตาราง

Operation	Singly	Doubly	Circular
Access First	$O(1)$	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$
Insert First	$O(1)$	$O(1)$	$O(1)$
Insert Last	$O(n)$	$O(1)$	$O(1)$
Delete First	$O(1)$	$O(1)$	$O(1)$

จากนั้นตอบว่า

หากต้องการสร้างระบบที่ต้องเดินข้อมูล ไปข้างหน้าและย้อนกลับบ่อย ๆ ควรเลือก Linked List แบบใด เพราะเหตุใด?

ตอบ: Doubly Linked List เพราะมีทั้ง next และ prev จึงเดินข้อมูลไปข้างหน้าและย้อนกลับได้สะดวก

ข้อ 3: ระบบตรวจสอบวงเล็บและ Undo

หัวข้อ: **Stack**

กำหนดข้อความ เช่น

{A + [B * (C + D)]}

ให้นักศึกษาใช้ Stack ตรวจสอบว่าเครื่องหมาย

()

[]

{}

เปิด-ปิดถูกต้องหรือไม่

ตัวอย่าง

Input:

{A + [B * (C + D)]}

Output:

Balanced

แต่

Input:

{A + [B * (C + D)]}

Output:

Not Balanced

จากนั้นให้เพิ่มส่วนที่ 2 คือ Undo Operation

กำหนดคำสั่ง

ADD A

ADD B

ADD C

UNDO

ADD D

UNDO

ให้ใช้ Stack เก็บประวัติการทำงาน

ผลลัพธ์ที่คาดหวัง เช่น

ADD A

Stack: A

ADD B

Stack: A B

ADD C

Stack: A B C

UNDO

Remove C

ADD D

Stack: A B D

UNDO

Remove D

คำถามวิเคราะห์

1. push() มี Time Complexity เท่าใด? ตอบ: $O(1)$
2. pop() มี Time Complexity เท่าใด? ตอบ: $O(1)$
3. ถ้ามีข้อความยาว n ตัวอักษร การตรวจสอบวงเล็บทั้งหมดมี Big-O เท่าใด? ตอบ: $O(n)$
4. ทำไม Stack จึงเหมาะกับปัญหา Matching Parentheses มากกว่า Queue?

ตอบ: เพราะ Stack ทำงานแบบ LIFO จึงตรวจวงเล็บตัวที่เปิดล่าสุดก่อน ซึ่งตรงกับลำดับการปิดวงเล็บ

ข้อ 4: ระบบคิวบริการที่มีระดับความสำคัญ

หัวข้อ: **Queue + Array + Linked List**

ศูนย์บริการคอมพิวเตอร์มีนักศึกษาสามารถรับบริการ แต่ละคนมี Student ID และปัญหาที่ต้องการรับบริการ
ตัวอย่างข้อมูล

65001 Password Reset

65002 WiFi Problem

65003 Software Installation

65004 Account Locked

65005 Printer Problem

ให้นักศึกษาสร้าง Queue และรองรับ operation

```
enqueue()  
dequeue()  
peek()  
displayQueue()
```

จากนั้นเกิดเหตุการณ์ดังนี้

1. 65001 เข้าคิว
2. 65002 เข้าคิว
3. 65003 เข้าคิว
4. ให้บริการ 1 คน
5. 65004 เข้าคิว
6. ให้บริการ 1 คน
7. 65005 เข้าคิว

ให้นักศึกษาแสดงสถานะ Queue ทุกขั้นตอน เช่น

Step 1

Front -> 65001

Step 2

Front -> 65001 -> 65002

Step 3

Front -> 65001 -> 65002 -> 65003

Step 4

Serve: 65001

Front -> 65002 -> 65003

5) +65004 : 65002 -> 65003 -> 65004

6) serve 65002 : 65003 -> 65004

7) +65005 : 65003 -> 65004 -> 65005

จากนั้นให้สร้าง Queue 2 วิธี

Version A: Array-based Queue

Version B: Linked-List-based Queue

แล้วเปรียบเทียบ

Operation	Array Queue	Linked List Queue
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Memory	Fixed	Dynamic

Challenge เพิ่มเติม

สมมติ Queue ใช้ Array ขนาด 5

[][][][][]

เมื่อมีการ dequeue() หลายครั้ง ต้องอธิบายว่าเหตุใด Circular Queue จึงดีกว่าการเลื่อนข้อมูลทุกครั้ง พร้อมวิเคราะห์

แบบเลื่อน Array ทุกครั้ง:

dequeue = $O(n)$

Circular Queue:

dequeue = $O(1)$

ตอบ:

แบบเลื่อน Array ทุกครั้ง: ต้องขยับข้อมูลที่เหลือไปด้านหน้า หลัง dequeue จึงใช้เวลา $O(n)$ และมีการย้ายข้อมูลทุกครั้ง

Circular Queue: เปลี่ยนตำแหน่ง front ไปช่องถัดไปด้วยการวนกลับ ไม่ต้องเลื่อนข้อมูล จึง dequeue ได้ $O(1)$

ดังนั้นเมื่อ dequeue หลายครั้ง Circular Queue ดีกว่า เพราะไม่ต้องย้ายข้อมูล ทำงานเร็วกว่า